
Host & Network Penetration Testing: Network-Based Attacks CTF 1

RedLine Stealer Traffic Analysis via Wireshark PCAP Forensics

Victim Host	10.7.10.47 (DESKTOP-9PEA63H)
Attack Type	Malware Traffic Analysis (RedLine Stealer)
Platform	INE Attack Defense Labs
Prepared	May 2026
Context	eJPT Exam Preparation

Contents

1	Goal and Context	2
2	Investigation Overview	2
3	Lab Setup and Environment	2
3.1	Analysis Environment	3
3.2	Why Wireshark for This Investigation	3
4	Background: RedLine Stealer	3
5	Flag 1: C2 Initial Beacon Domain	4
6	Flag 2: Victim IP and MAC Address	5
7	Flag 3: Victim Hostname via NBNS	5
8	Flag 4: Infected User from mystery_file.ps1 Path	6
9	Flag 5: PowerShell User-Agent String	7
10	Flag 6: Coinbase Wallet Extension ID	8
11	Key Takeaways	9

1 Goal and Context

This lab presents a captured network packet file (`test.pcap`) from a RedLine Stealer infection and asks six forensic questions answered entirely through Wireshark. There is no exploitation involved: the work is packet analysis, filter construction, and byte-level investigation of malware command-and-control traffic. The six flags collectively build a complete picture of the infection: which site the malware contacted, which host and user were affected, how the malware identified itself in HTTP traffic, and what sensitive data the malware was designed to steal.

RedLine Stealer is a credential and cryptocurrency harvesting malware that has been commercially sold on underground forums since 2020. It targets browser-saved passwords, session cookies, cryptocurrency wallet browser extensions, and desktop files matching patterns like `*.wallet` and `seed*`. Understanding how to read its traffic in a PCAP is directly applicable to incident response work: a SOC analyst who can identify RedLine's C2 traffic pattern, locate the infected host's identity, and enumerate what data was targeted has the skills needed to scope a real breach.

2 Investigation Overview

Investigation Overview

- Flag 1 (C2 Domain):** Applied `http.response.code == 200` to isolate successful HTTP responses. Read the Request URI from the packet details to identify `623start.site`.
- Flag 2 (Victim IP and MAC):** Identified the infected client as destination `10.7.10.47` with MAC `80:86:5b:ab:1e:c4` from the Ethernet II layer of the same HTTP response packet.
- Flag 3 (Hostname via NBNS):** Applied the `nbns` filter to surface NetBIOS Name Service registration broadcasts. Read the hostname `DESKTOP-9PEA63H` from the Queries or Additional records section of any NBNS packet from `10.7.10.47`.
- Flag 4 (Infected User):** Used Ctrl+F with Packet bytes and searched for `mystery_file.ps1`. Read the full path from the highlighted packet bytes: `C:\Users\rwalters\Documents\mystery_file.ps1`, identifying user `rwalters`.
- Flag 5 (PowerShell User-Agent):** Used Ctrl+F with Packet details and searched for `powershell`. Expanded the HTTP header in the matching packet to read the User-Agent field, which identified `WindowsPowerShell` as the flag component.
- Flag 6 (Coinbase Wallet Extension ID):** Used Ctrl+F with Packet bytes and searched for `coinbase`. Right-clicked the bytes pane and selected "as Printable Text" to render the wallet target list. Located the Coinbase entry: `hnfanknocfeofbddgcijnmhnfnkdnaad`.

3 Lab Setup and Environment

3.1 Analysis Environment

PCAP File	test.pcap
Analysis Tool	Wireshark
Victim IP	10.7.10.47
Victim MAC	80:86:5b:ab:1e:c4
Victim Hostname	DESKTOP-9PEA63H
Infected User	rwalters
C2 Server IP	194.26.135.119 (TCP port 12432)
Malware Family	RedLine Stealer
Screenshot Count	12

3.2 Why Wireshark for This Investigation

Approach and Toolset

All six flags are contained within a single PCAP file and are recoverable without any external tools. Wireshark’s display filters isolate traffic by protocol and response code. Its Ctrl+F byte-level search locates specific strings buried inside encrypted-looking or binary TCP streams. The “as Printable Text” copy option converts raw bytes into readable ASCII, which is essential for the C2 stream where the wallet target list is transmitted as part of the malware’s tasking data. This investigation pattern: filter, inspect, search, extract, is the same one used in real incident response when a PCAP is provided as evidence.

4 Background: RedLine Stealer

RedLine Stealer is a commodity infostealer sold as malware-as-a-service on Russian-language cybercrime forums. It was first observed in early 2020 and has remained one of the most frequently encountered stealers in threat intelligence reporting. A buyer receives a builder that generates a customised binary, which they then distribute through phishing, malvertising, or fake software downloads.

Once running on a victim host, RedLine exfiltrates browser-saved credentials, autofill data, session cookies, and cryptocurrency wallet data. Its wallet targeting works by searching for Chromium-based browser extensions by their extension ID strings, as well as scanning the filesystem for wallet file patterns. The malware communicates with its C2 server over a custom TCP protocol on non-standard ports, typically in the 10000–65000 range. The initial beacon often includes an HTTP request to a staging server that tracks infection status before the main C2 channel is established.

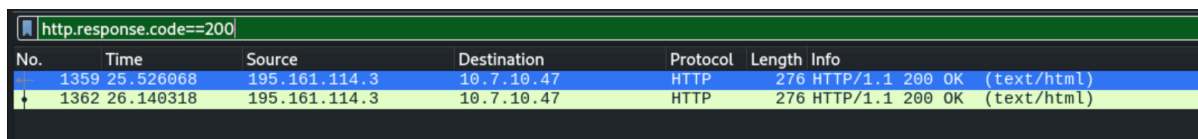
For defenders, RedLine’s initial HTTP beacon is often the most visible indicator. The User-Agent string used by the PowerShell dropper is distinctive, and the C2 traffic to a non-standard TCP port from a workstation is anomalous in most enterprise environments. Network-level detection does not require decrypting the traffic: the port, destination IP, and volume of outbound data are sufficient to trigger investigation.

5 Flag 1: C2 Initial Beacon Domain

Why Filter on HTTP 200 OK

RedLine's initial HTTP contact reports the infection status to a staging server and receives a task configuration in return. The server responds with a 200 OK. Filtering for `http.response.code == 200` isolates successful HTTP exchanges and eliminates redirects, errors, and other noise, leaving only the packets where a server actively responded to the infected host.

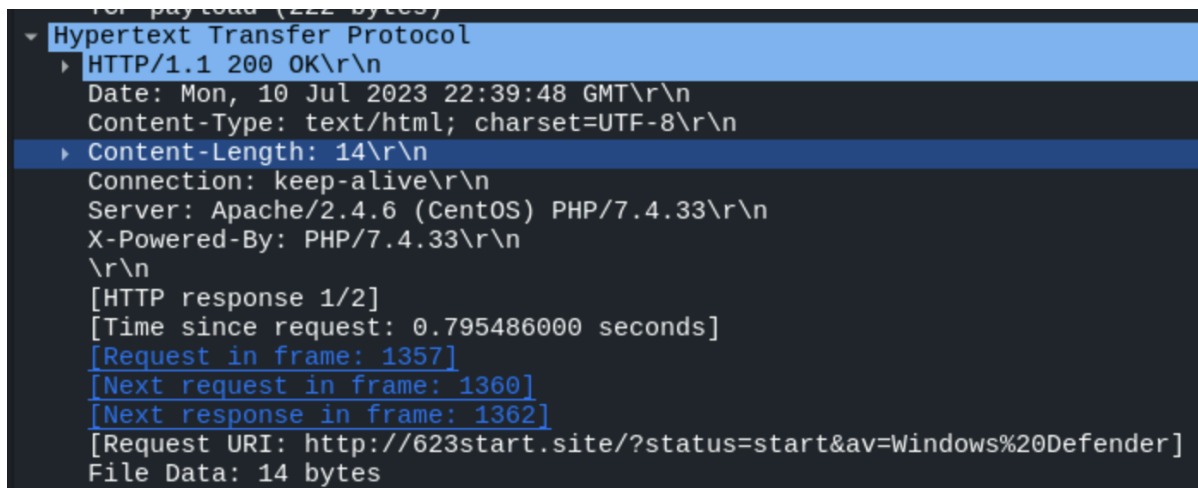
```
http.response.code == 200
```



No.	Time	Source	Destination	Protocol	Length	Info
1359	25.526068	195.161.114.3	10.7.10.47	HTTP	276	HTTP/1.1 200 OK (text/html)
1362	26.140318	195.161.114.3	10.7.10.47	HTTP	276	HTTP/1.1 200 OK (text/html)

Figure 1. Filter `http.response.code == 200` returns two packets: frames 1359 and 1362, both HTTP/1.1 200 OK responses from 195.161.114.3 to the victim at 10.7.10.47.

Selecting the first result and expanding the Hypertext Transfer Protocol section in the packet details pane revealed the full request URI.



Hypertext Transfer Protocol	
HTTP/1.1 200 OK\r\n	
Date: Mon, 10 Jul 2023 22:39:48 GMT\r\n	
Content-Type: text/html; charset=UTF-8\r\n	
Content-Length: 14\r\n	
Connection: keep-alive\r\n	
Server: Apache/2.4.6 (CentOS) PHP/7.4.33\r\n	
X-Powered-By: PHP/7.4.33\r\n	
\r\n	
[HTTP response 1/2]	
[Time since request: 0.795486000 seconds]	
[Request in frame: 1357]	
[Next request in frame: 1360]	
[Next response in frame: 1362]	
[Request URI: http://623start.site/?status=start&av=Windows%20Defender]	
File Data: 14 bytes	

Figure 2. The HTTP response detail shows the Request URI field: `http://623start.site/?status=start&av=Windows%20Defender`, confirming `623start.site` as the domain contacted by the malware. The `av=Windows%20Defender` parameter reports the victim's active antivirus product to the C2.

Flag 1

Question: What is the domain name accessed by the infected user that returned a 200 OK response code?

Wireshark filter: `http.response.code == 200`

Answer: `623start.site`

6 Flag 2: Victim IP and MAC Address

Why Read from the Same HTTP Response Packet

The HTTP 200 OK packets are sent from the C2 server to the infected host. The destination IP and MAC address in those packets belong to the victim. Reading them from the Ethernet II and IPv4 layers of the same packet used for Flag 1 avoids any ambiguity about which host is infected.

With the same packet selected, I expanded the Ethernet II layer in the packet details pane.

```

▼ Ethernet II, Src: Cisco_98:ad:54 (00:b0:64:98:ad:54), Dst: 80:86:5b:ab:1e:c4
  ▼ Destination: 80:86:5b:ab:1e:c4 (80:86:5b:ab:1e:c4)
    Address: 80:86:5b:ab:1e:c4 (80:86:5b:ab:1e:c4)
    ....0. .... = LG bit: Globally unique address (factor
    ....0. .... = IG bit: Individual address (unicast)
  ▼ Source: Cisco_98:ad:54 (00:b0:64:98:ad:54)
    Address: Cisco_98:ad:54 (00:b0:64:98:ad:54)
    ....0. .... = LG bit: Globally unique address (factor
    ....0. .... = IG bit: Individual address (unicast)
  Type: IPv4 (0x0800)
▼ Internet Protocol Version 4, Src: 195.161.114.3, Dst: 10.7.10.47
  0100 .... = Version: 4
  ....0101 = Header Length: 20 bytes (5)
  ▶ Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
  Total Length: 262
  Identification: 0xb190 (45456)

```

Figure 3. The Ethernet II layer shows Destination MAC 80:86:5b:ab:1e:c4 (the victim). The Internet Protocol layer confirms Destination IP 10.7.10.47. The source is a Cisco device (00:b0:64:98:ad:54), which is the gateway forwarding traffic from the external C2 server.

Flag 2

Question: What is the IP address and MAC address of the infected Windows client?

Answer: IP 10.7.10.47, MAC 80:86:5b:ab:1e:c4

7 Flag 3: Victim Hostname via NBNS

Why NBNS Reveals the Hostname Without Authentication

NetBIOS Name Service (NBNS) operates on UDP port 137. Windows hosts broadcast registration packets to announce their hostname on the local network. These broadcasts are unauthenticated and contain the machine name in plaintext. A host captured in a PCAP will typically send multiple NBNS registration packets at startup, making the hostname easy to recover from any capture taken early in a session.

```
nbns
```

nbns						
No.	Time	Source	Destination	Protocol	Length	Info
667	3.200950	10.7.10.47	10.7.10.255	NBNS	110	Registration NB DESKTOP-9PEA63H<20>
664	2.435296	10.7.10.47	10.7.10.255	NBNS	110	Registration NB DESKTOP-9PEA63H<20>
581	1.658373	10.7.10.47	10.7.10.255	NBNS	110	Registration NB DESKTOP-9PEA63H<20>
320	0.908673	10.7.10.47	10.7.10.255	NBNS	110	Registration NB DESKTOP-9PEA63H<20>
666	2.577040	10.7.10.47	10.7.10.255	NBNS	110	Registration NB DESKTOP-9PEA63H<00>
582	1.798931	10.7.10.47	10.7.10.255	NBNS	110	Registration NB DESKTOP-9PEA63H<00>
352	1.033298	10.7.10.47	10.7.10.255	NBNS	110	Registration NB DESKTOP-9PEA63H<00>
19	0.289112	10.7.10.47	10.7.10.255	NBNS	110	Registration NB DESKTOP-9PEA63H<00>
665	2.576938	10.7.10.47	10.7.10.255	NBNS	110	Registration NB COOLWEATHERCOAT<00>
583	1.799035	10.7.10.47	10.7.10.255	NBNS	110	Registration NB COOLWEATHERCOAT<00>
353	1.033338	10.7.10.47	10.7.10.255	NBNS	110	Registration NB COOLWEATHERCOAT<00>
20	0.289365	10.7.10.47	10.7.10.255	NBNS	110	Registration NB COOLWEATHERCOAT<00>

Figure 4. Filter `nbns` shows multiple NBNS Registration NB packets from 10.7.10.47 to the broadcast address 10.7.10.255. The Info column shows two distinct names: DESKTOP-9PEA63H (the machine hostname, appearing with both <20> Server service and <00> Workstation service suffixes) and COOLWEATHERCOAT (the workgroup or domain name).

Selecting any packet showing DESKTOP-9PEA63H and expanding the Additional records section confirmed the hostname in the Name field.

```

  ▾ Additional records
    ▾ DESKTOP-9PEA63H<20>: type NB, class IN
      Name: DESKTOP-9PEA63H<20> (Server service)
      Type: NB (32)
      Class: IN (1)
      Time to live: 3 days, 11 hours, 20 minutes
      Data length: 6

```

Figure 5. The Additional records section of the NBNS packet shows Name: DESKTOP-9PEA63H<20> (Server service), confirming the victim machine's hostname.

Flag 3

Question: Which Wireshark filter determines the victim's hostname from NBNS traffic, and what is the hostname?

Wireshark filter: `nbns`

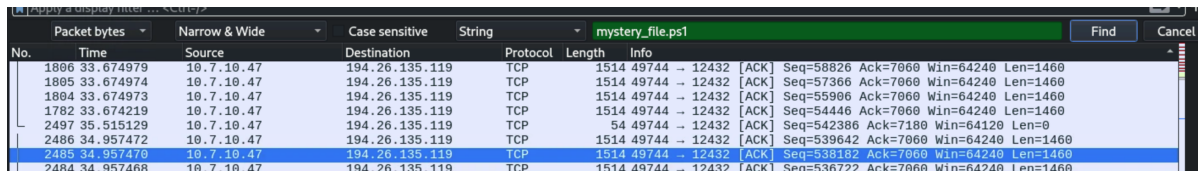
Answer: DESKTOP-9PEA63H

8 Flag 4: Infected User from `mystery_file.ps1` Path

Why Search Packet Bytes for the Script Name

The script name `mystery_file.ps1` is not visible as an HTTP parameter or DNS name. It appears inside the binary content of a C2 TCP stream, where the malware transmitted process information back to its operator. Display filters cannot search raw byte content. The Packet bytes search mode in Ctrl+F scans every byte of every packet for the target string and highlights the matching packet in the packet list.

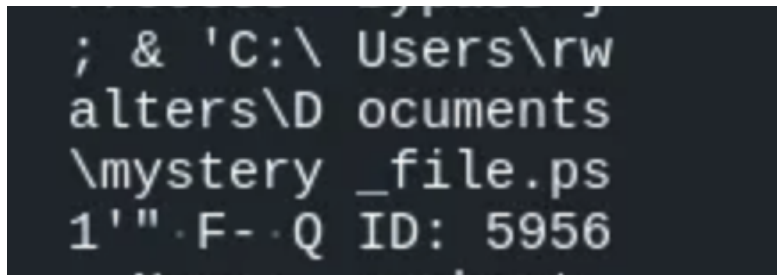
I cleared all display filters, opened Ctrl+F, set the left dropdown to **Packet bytes**, set the type to **String**, and searched for `mystery_file.ps1`.



No.	Time	Source	Destination	Protocol	Length	Info
1886	33.674979	10.7.10.47	194.26.135.119	TCP	1514	49744 → 12432 [ACK] Seq=59826 Ack=7060 Win=64240 Len=1460
1895	33.674974	10.7.10.47	194.26.135.119	TCP	1514	49744 → 12432 [ACK] Seq=57366 Ack=7060 Win=64240 Len=1460
1894	33.674973	10.7.10.47	194.26.135.119	TCP	1514	49744 → 12432 [ACK] Seq=55906 Ack=7060 Win=64240 Len=1460
1782	33.674219	10.7.10.47	194.26.135.119	TCP	1514	49744 → 12432 [ACK] Seq=54446 Ack=7060 Win=64240 Len=1460
2497	35.515129	10.7.10.47	194.26.135.119	TCP	54	49744 → 12432 [ACK] Seq=542386 Ack=7180 Win=64120 Len=0
2486	34.957472	10.7.10.47	194.26.135.119	TCP	1514	49744 → 12432 [ACK] Seq=539642 Ack=7060 Win=64240 Len=1460
2485	34.957470	10.7.10.47	194.26.135.119	TCP	1514	49744 → 12432 [ACK] Seq=538182 Ack=7060 Win=64240 Len=1460
2484	34.957468	10.7.10.47	194.26.135.119	TCP	1514	49744 → 12432 [ACK] Seq=536722 Ack=7060 Win=64240 Len=1460

Figure 6. Ctrl+F with Packet bytes and String mode, searching `mystery_file.ps1`, highlights frame 2485: a TCP packet from 10.7.10.47 to C2 server 194.26.135.119 on port 12432.

With the packet highlighted, I looked at the bytes panel at the bottom of the Wireshark window, where the path containing the script name was visible.



```

; & 'C:\Users\rwalters\Documents
\mystery_file.ps
1'" -F- -Q ID: 5956

```

Figure 7. The bytes panel of the highlighted packet shows the partial path: `C:\Users\rwalters\Documents\mystery_file.ps1`, identifying `rwalters` as the user whose session ran the script.

Flag 4

Question: Which user got infected and ran the `mystery_file.ps1` PowerShell script?

Method: Ctrl+F, Packet bytes, String, search `mystery_file.ps1`

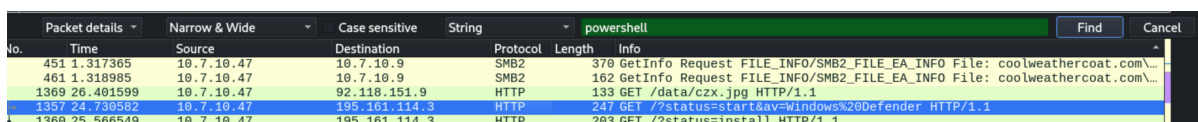
Answer: `rwalters`

9 Flag 5: PowerShell User-Agent String

Why Packet Details Search Locates the User-Agent

The HTTP User-Agent header is a parsed field in Wireshark's dissection of HTTP packets. Searching in Packet details mode matches against the human-readable field names and values that Wireshark displays in the middle pane, rather than the raw bytes. This is faster for finding specific HTTP header values because it searches the already-parsed field text.

I opened Ctrl+F, set the left dropdown to **Packet details**, kept the type as **String**, and searched for `powershell`.



No.	Time	Source	Destination	Protocol	Length	Info
451	1.317365	10.7.10.47	10.7.10.9	SMB2	370	GetInfo Request FILE_INFO/SMB2_FILE_EA_INFO File: coolweathercoat.com\...
451	1.318985	10.7.10.47	10.7.10.9	SMB2	162	GetInfo Request FILE_INFO/SMB2_FILE_EA_INFO File: coolweathercoat.com\...
1369	26.481599	10.7.10.47	92.118.151.9	HTTP	133	GET /data/czx.jpg HTTP/1.1
1357	24.730582	10.7.10.47	195.161.114.3	HTTP	247	GET /?status=start&av=Windows%20Defender HTTP/1.1
1368	25.566549	10.7.10.47	195.161.114.3	HTTP	203	GET /?status=install HTTP/1.1

Figure 8. Packet details search for `powershell` highlights frame 1357: a GET request from 10.7.10.47 to 195.161.114.3, matching `GET /?status=start&av=Windows%20Defender HTTP/1.1`.

Expanding the Hypertext Transfer Protocol section in the details pane of the highlighted packet revealed the full User-Agent header.


```

Hypertext Transfer Protocol
GET /?status=start&av=Windows%20Defender HTTP/1.1\r\n
User-Agent: Mozilla/5.0 (Windows NT; Windows NT 10.0; en-US) WindowsPowerShell/5.1.19041.3031\r\n
Host: 623start.site\r\n
Connection: Keep-Alive\r\n

```

Figure 9. The HTTP request header shows: User-Agent: Mozilla/5.0 (Windows NT; Windows NT 10.0; en-US) WindowsPowerShell/5.1.19041.3031. The identifying flag component is WindowsPowerShell.

Flag 5

Question: What User-Agent string indicates traffic generated by a PowerShell script?

Method: Ctrl+F, Packet details, String, search powershell

Full User-Agent: Mozilla/5.0 (Windows NT; Windows NT 10.0; en-US) WindowsPowerShell/5.1.19041.3031

Answer (flag): WindowsPowerShell

10 Flag 6: Coinbase Wallet Extension ID

Why the Wallet List Is in Raw TCP Bytes

RedLine transmits its target configuration, including the list of cryptocurrency wallet browser extensions it is searching for, as part of the C2 communication. This data is not transmitted over HTTP and does not appear as a parsed protocol field. It exists only in the raw bytes of a TCP stream. The Packet bytes search locates the packet containing the string “coinbase”, and the “as Printable Text” copy option renders the surrounding bytes as readable ASCII, making the extension ID visible.

I opened Ctrl+F, set the left dropdown to **Packet bytes**, kept the type as **String**, and searched for **coinbase**.

No.	Time	Source	Destination	Protocol	Length	Info
1750	33.673822	194.26.135.119	10.7.10.47	TCP	54	12432 → 49744 [ACK] Seq=7060 Ack=10646 Win=64240 Len=0
1896	33.676078	194.26.135.119	10.7.10.47	TCP	54	12432 → 49744 [ACK] Seq=7060 Ack=105546 Win=64240 Len=0
1895	33.676046	194.26.135.119	10.7.10.47	TCP	54	12432 → 49744 [ACK] Seq=7060 Ack=104886 Win=64240 Len=0
1894	33.676015	194.26.135.119	10.7.10.47	TCP	54	12432 → 49744 [ACK] Seq=7060 Ack=102626 Win=64240 Len=0
1893	33.675984	194.26.135.119	10.7.10.47	TCP	54	12432 → 49744 [ACK] Seq=7060 Ack=101166 Win=64240 Len=0
1711	28.498640	194.26.135.119	10.7.10.47	TCP	1514	12432 → 49744 [ACK] Seq=4541 Ack=426 Win=64240 Len=1460
1710	28.498638	194.26.135.119	10.7.10.47	TCP	1514	12432 → 49744 [ACK] Seq=3081 Ack=426 Win=64240 Len=1460
1704	27.984276	194.26.135.119	10.7.10.47	TCP	54	12432 → 49744 [ACK] Seq=2 Ack=260 Win=64240 Len=0

Figure 10. Packet bytes search for **coinbase** highlights frame 1710: a TCP packet from C2 server 194.26.135.119 to victim 10.7.10.47 on port 12432, part of the malware’s task configuration download.

With the packet highlighted, I right-clicked the bytes panel and selected **as Printable Text** to render the content as readable ASCII.

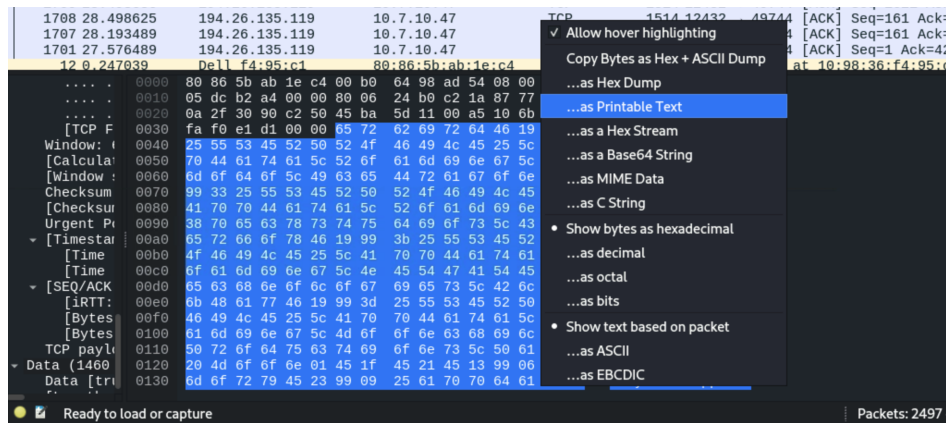


Figure 11. Right-clicking the bytes panel of the highlighted packet shows the context menu with “as Printable Text” selected, which converts the raw binary C2 stream into human-readable ASCII output.

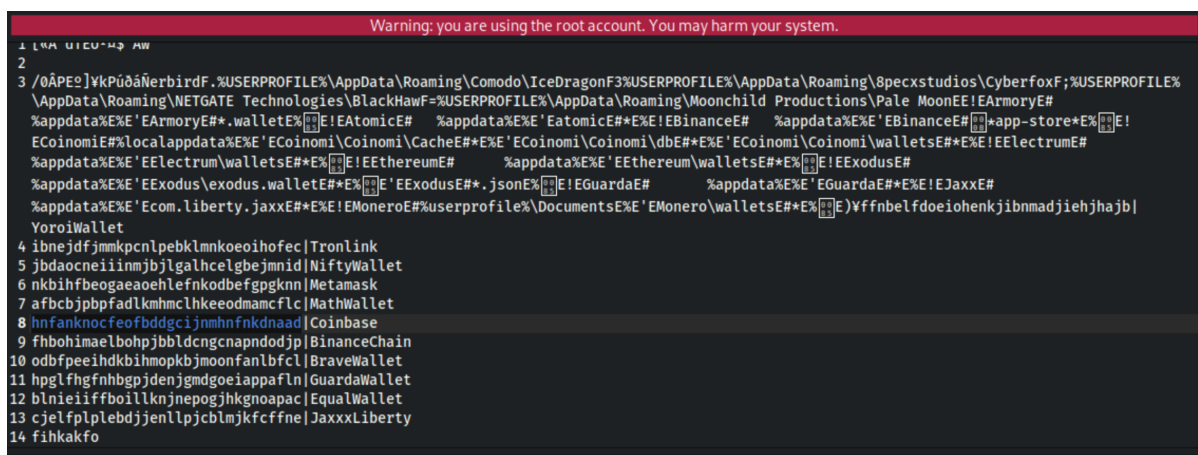


Figure 12. The printable text view of the C2 configuration stream shows a wallet extension list. Line 8 (highlighted in the screenshot) reads: `hnfanknocfeofbddgcijnmhfnkdnaad|Coinbase`, identifying the Coinbase wallet browser extension ID. Other visible entries include Metamask (`nkbihfbeogaeaoehlefnkodbefgpgknn`), Tronlink (`ibnejdfjmmkpcnlpebklmnkoeiohofec`), and BinanceChain (`fhhohimaelbohpbjbbldcngcnapndodjp`).

Flag 6

Question: Which wallet extension ID is associated with the Coinbase wallet?

Method: Ctrl+F, Packet bytes, String, search `coinbase`. Right-click bytes pane, select “as Printable Text”.

Answer: `hnfanknocfeofbddgcijnmhfnkdnaad`

11 Key Takeaways

Key Takeaways

- **Display filters and byte searches solve different problems.** Display filters like `http.response.code == 200` and `nbns` isolate traffic by protocol and field value. Ctrl+F with Packet bytes finds arbitrary strings buried in binary TCP streams. Knowing which tool to reach for based on where the data lives is the core skill this lab develops.
- **NBNS registration packets leak the hostname without any active query.** A Win-

dows host announces its name on the network automatically. Any PCAP captured on the same segment will contain this information. For a defender or analyst, this means the victim's machine identity is often recoverable even from a partial capture.

- **RedLine's initial HTTP beacon carries the victim's antivirus product name as a URL parameter.** The request `?status=start&av=Windows%20Defender` tells the C2 operator which security software is present before any payload is delivered. This kind of beacon is a detectable indicator: an outbound HTTP GET to an unfamiliar domain with `av=` in the query string is anomalous in any enterprise environment.
- **The PowerShell User-Agent is distinctive and does not match any legitimate browser.** The string `WindowsPowerShell` in a User-Agent header means the HTTP request was made by a PowerShell script, not a browser. Alerting on this User-Agent in a web proxy or firewall log is a low-cost, high-value detection rule.
- **The wallet target list in the C2 stream shows exactly what RedLine was looking for.** The extension IDs cover over a dozen wallets. An analyst who can extract and enumerate this list from a PCAP can tell an affected organisation precisely which wallet software was at risk, which is directly relevant to scoping the business impact of the breach.
- **"as Printable Text" is an underused Wireshark feature.** When a TCP stream carries semi-structured data mixed with binary encoding, copying as printable text strips the unprintable bytes and leaves the human-readable portions. It is faster than following the full TCP stream when you only need the content of a single packet.
- **The C2 port (12432) is non-standard and immediately suspicious.** An outbound TCP connection from a workstation to a non-CDN, non-cloud IP on a high-numbered port is an anomaly worth investigating in any network. This lab reinforces that protocol awareness, knowing what ports are normal for a given environment, is a prerequisite for effective traffic analysis.